

基于蒙特卡洛模拟的种植策略优化模型

摘 要

作为国民经济的重要支柱，农业的可持续发展是实现乡村振兴和保障国家粮食安全的关键路径。本文结合蒙特卡洛模拟和非线性规划模型，旨在制定最优种植策略，以最大化耕地资源利用和农业生产效益。

针对问题一，首先进行数据预处理和多角度可视化分析，确定各项数据预期值：其中销售单价取区间均值；预期销售量用 2023 年对应作物的季总产量表示。其次，分别定义单季作物和两季作物的总收益计算方式。进而，针对作物超产部分滞销和半价出售两种情况，分别构建以七年总收益最大化为目标的非线性规划模型，引入二进制变量进行求解，得到最大收益分别为 34623417.97 和 85817862.12。最后对模型进行优化调整以适应销售价格波动和实际供需关系。

针对问题二，本文结合文献确定预期销售量、亩产量、种植成本和销售价格分别服从不同的概率分布。基于此进一步优化非线性规划模型，引入风险调整系数 r_i^k 和蒙特卡洛方法，模拟极端天气影响，求解最优种植方案并对收益分布及其波动情况进行可视化分析。结果显示波动较为稳定，说明该最优种植方案具有良好适应性和抗风险能力。

针对问题三，本文通过皮尔逊相关系数量化预期销售量、销售价格、种植成本之间的相关性。结果显示销售单价均值和种植成本呈较强正相关、预期销售量和种植成本相关性较弱、预期销售量和销售单价均值呈较强负相关，符合现实情况。我们进一步将相关性分析应用于简化模拟数据。最后，引入替代效应和互补性修正因子，构建优化模型求解最优种植策略，结果显示相较于问题二，问题三的策略提高了总收益并且优化了成本控制和风险管理。

最后，本文对所建立的模型进行分析，综合评价模型的优缺点，并探讨了模型在其他领域推广应用的潜力。

关键词：非线性规划 蒙特卡洛模拟 优化问题 皮尔逊相关系数

一、问题重述

1.1 问题背景

随着全球人口的不断增长和生态资源的日益紧张，农业可持续发展成为乡村经济振兴的重要目标^[1]。作为农业生产与发展的基础，耕地资源的合理利用具有重要现实意义。

农业的稳定发展离不开农作物的良好生态^[2]。为充分利用有限的耕地资源，提高农业生产效益并降低种植风险，必须考虑农作物的生长规律，结合土壤管理和种植技术，优化农作物种植策略，因地制宜发展有机种植产业，为国家粮食安全之路保驾护航。

1.2 问题要求

附件 1、2 给出了华北山区某乡村现有耕地和农作物基本情况、该乡村 2023 年农作物种植和相关统计数据。该乡村常年温度较低，拥有四种类型的露天耕地、普通大棚、智慧大棚。已知不同地块种植情况各异，且同一地块每季可合种不同农作物。为提高该乡村的农业生产效益，探索最优农作物种植方案，现需结合实际情况建立数学模型，分析下列问题：

问题一：以附件 2 中各项数据作为预期值，对于某种作物季总产量超出预期销售量的部分，按照如下两种处理方式，分别制定出 2024~2030 年最优种植方案：

- (1) 超产部分滞销；
- (2) 超产部分半价出售。

问题二：考虑小麦和玉米预期销售量年增长率为 5%~10%，其他作物预期年销售量和亩产量分别有 $\pm 5\%$ 和 $\pm 10\%$ 的波动，种植成本每年增加 5%，不同作物的价格有不同的变化趋势，综合考虑上述不确定性因素及潜在风险，制定 2024~2030 年农作物的最优种植方案。

问题三：问题二的基础上，综合考虑作物之间的替代性、互补性及其与销售量、价格、成本之间的相关性，给出最优种植策略，并与问题二的结果进行对比分析。

二、问题分析

2.1 问题一的分析

问题一要求对作物超产部分进行两种处理，分别制定未来七年的最优农作物种植方案。

首先要进行数据预处理，并从多种角度进行数据可视化分析。其次，要确定未来七年各项相关数据的预期值。接着，针对单季作物和两季作物，要分别考虑总收益的计算方式：对于单季作物，直接计算一年内总收益；对于两季作物，将两季的利润求和得到总收益。

最后，以未来七年的总收益最大化为目标，构建非线性规划模型。针对超产部分滞销和超产部分半价出售两种情况，确定不同的目标函数和约束条件，引入二进制变量进行模型求解。另外，可以考虑销售价格微小波动和具体供需关系，对模型优化调整，以制定更加稳定的种植策略。

2.2 问题二的分析

问题二的核心在于求解含有随机变量的最优种植方案。

首先,需要基于问题一的非线性规划模型,得到新的目标函数与约束条件。其次,需要考虑不确定性及潜在风险,可以通过查找文献并结合题目信息,确定预期销售量、亩产量、种植成本、销售价格四种随机变量服从的分布。由于这是一个涉及随机波动变量的复杂优化问题,难以使用确定性的方法解决,可以考虑使用蒙特卡洛模拟方法求解最优种植方案,并对收益分布及波动情况进行可视化分析。

2.3 问题三的分析

问题三要求综合考虑可替代性、互补性及相关性,制定未来七年的最优种植策略。

首先,引入皮尔逊相关系数,求解预期销售量、销售价格、种植成本三个变量之间的相关性。其次,引入替代效应修正因子 θ_{jm} 和互补性修正因子 φ_{jm} ,基于问题二的非线性规划模型,考虑农作物之间的替代性和互补性,确定新的目标函数及约束条件。

文章总体思路如图所示:

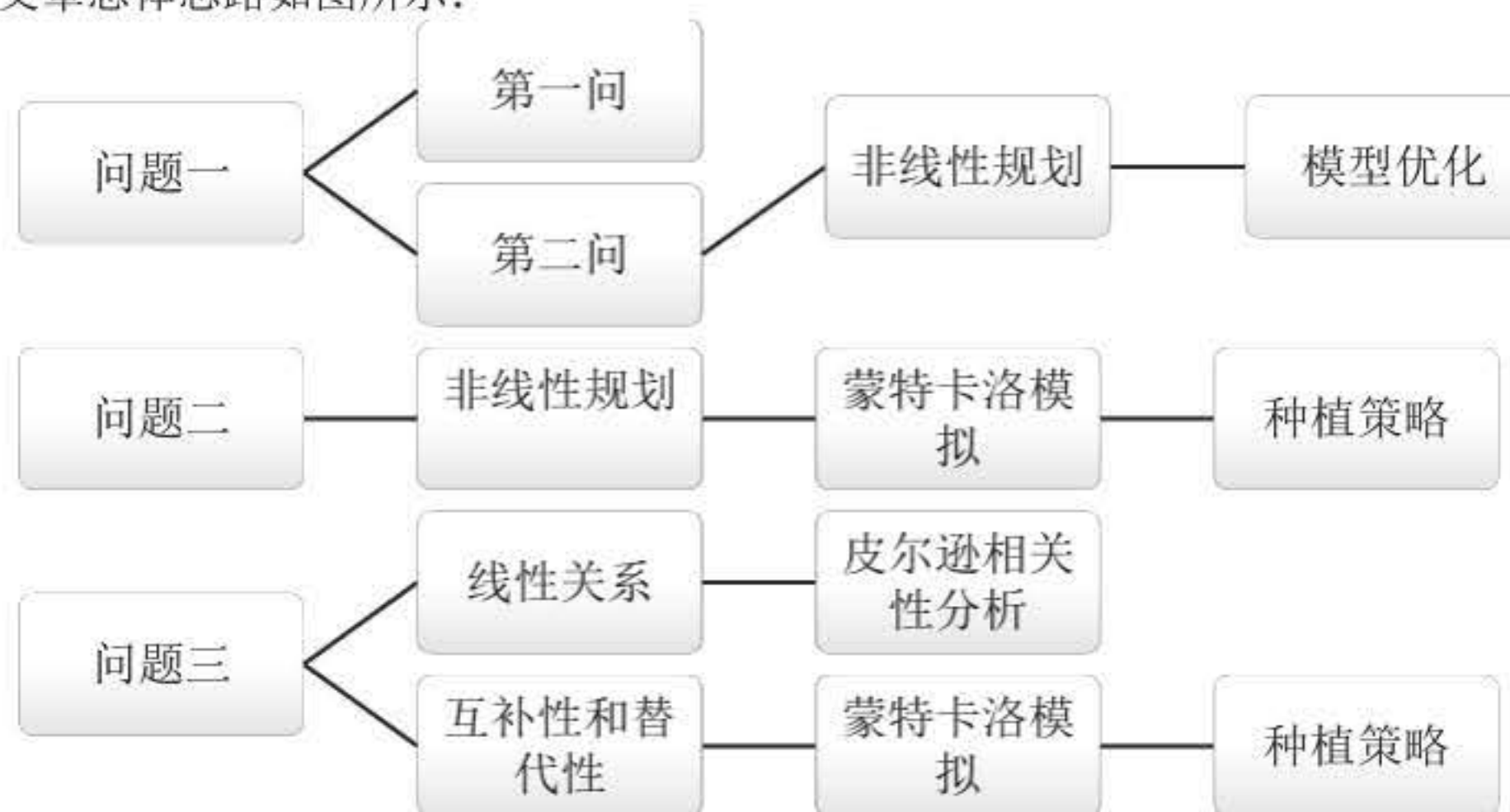


图 1 总体思路图

三、模型假设

- 1、假设问题一中各种农作物的预期销售量、种植成本、亩产量和销售单价相对于 2023 年保持稳定,近似认为种植成本、亩产量对应相等。
- 2、假设问题一中 2023 年达到供需平衡,各种农作物预期销售量用 2023 年对应季度总产量表示。
- 3、假设问题一中各种农作物预期销售单价为 2023 年销售单价区间的均值。
- 4、假设问题二、三中粮食类作物销售价格保持不变。

5、假设发生干旱、沙尘暴、暴雪、暴雨四种极端情况的概率相等。

四、符号说明

符号	说明
X_{ij}^k	第 k 年第 i 种地块上第 j 种作物的种植面积
N_{ij}	第 j 种作物在第 i 种地块上的亩产量
P_j	第 j 种作物的销售单价
C_{ij}	第 j 种作物在第 i 种地块上的种植成本
Y_j	第 j 种作物的预期总销量
X_i	第 i 种地块的总面积
W_i^k	第 k 年第 i 种地块的种植总收益
W	2024~2030 年的种植总收益
δ	每个地块的最小种植面积系数
S_{ij}	作物的市场供需调整系数（供大于需， $S_{ij} < 1$ ；供小于需， $S_{ij} > 1$ ）
Z_{ij}^k	二进制变量，表示第 k 年第 i 块地上是否种植第 j 种作物（1 表示种植，0 表示未种植）
r_i^k	风险调整系数
μ	季节性因素系数
θ_{jm}	替代效应修正因子
φ_{jm}	互补效应修正因子
k_1	第 k 年第一季种植
k_2	第 k 年第二季种植

五、 问题一模型的建立与求解

5.1 数据预处理及可视化分析

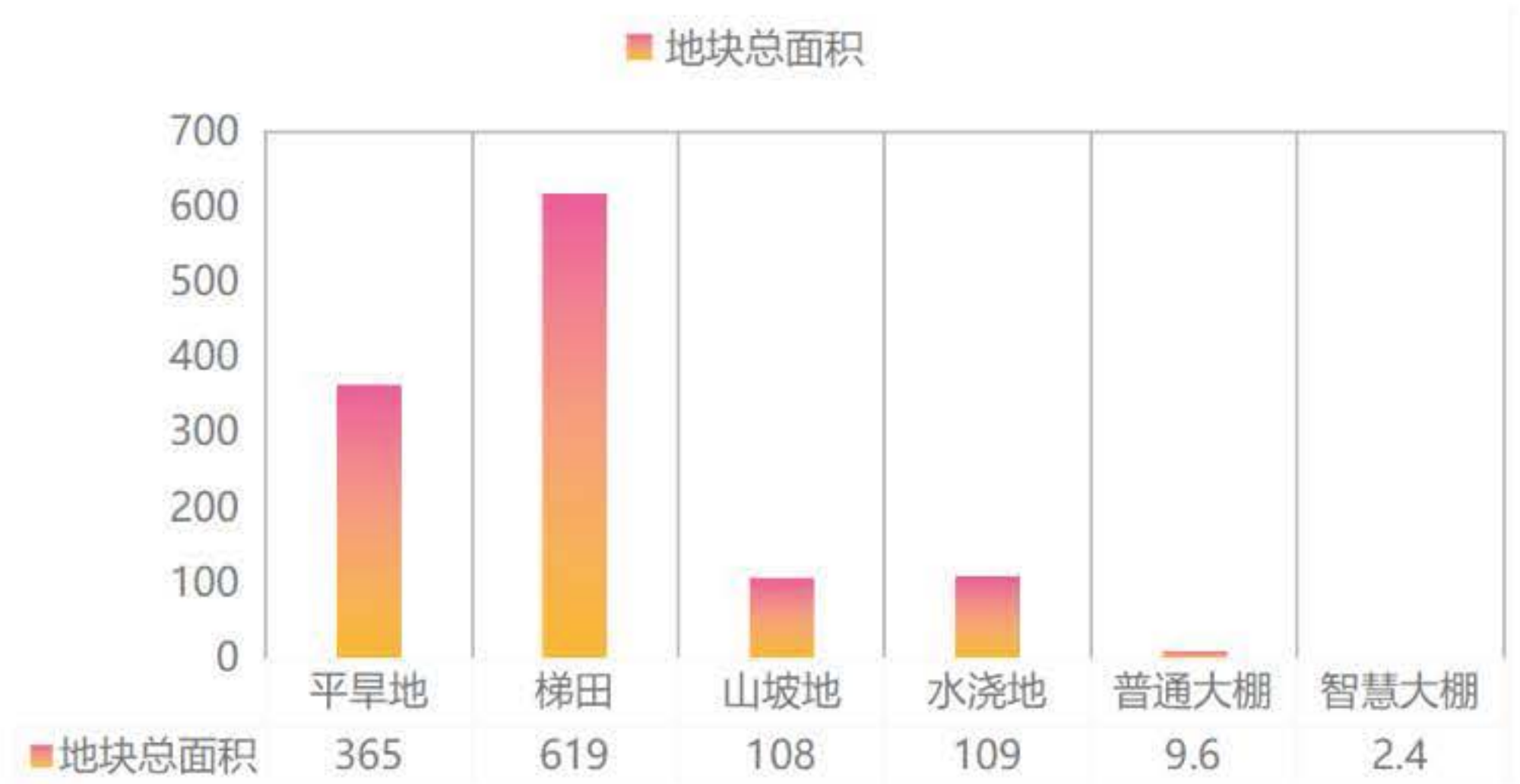


图 2 不同类型地块的总面积

在分析附件一、二的数据时，发现部分脱离实际的异常值，属于无效数据，故进行剔除处理。接着，从不同角度对数据进行可视化分析。

首先，对不同类型地块的总面积进行可视化，结果如图 2 所示。由图 2 可以看出，自然形成的地块面积较大，而人工制造的大棚面积较小，总面积仅达到 12 亩。

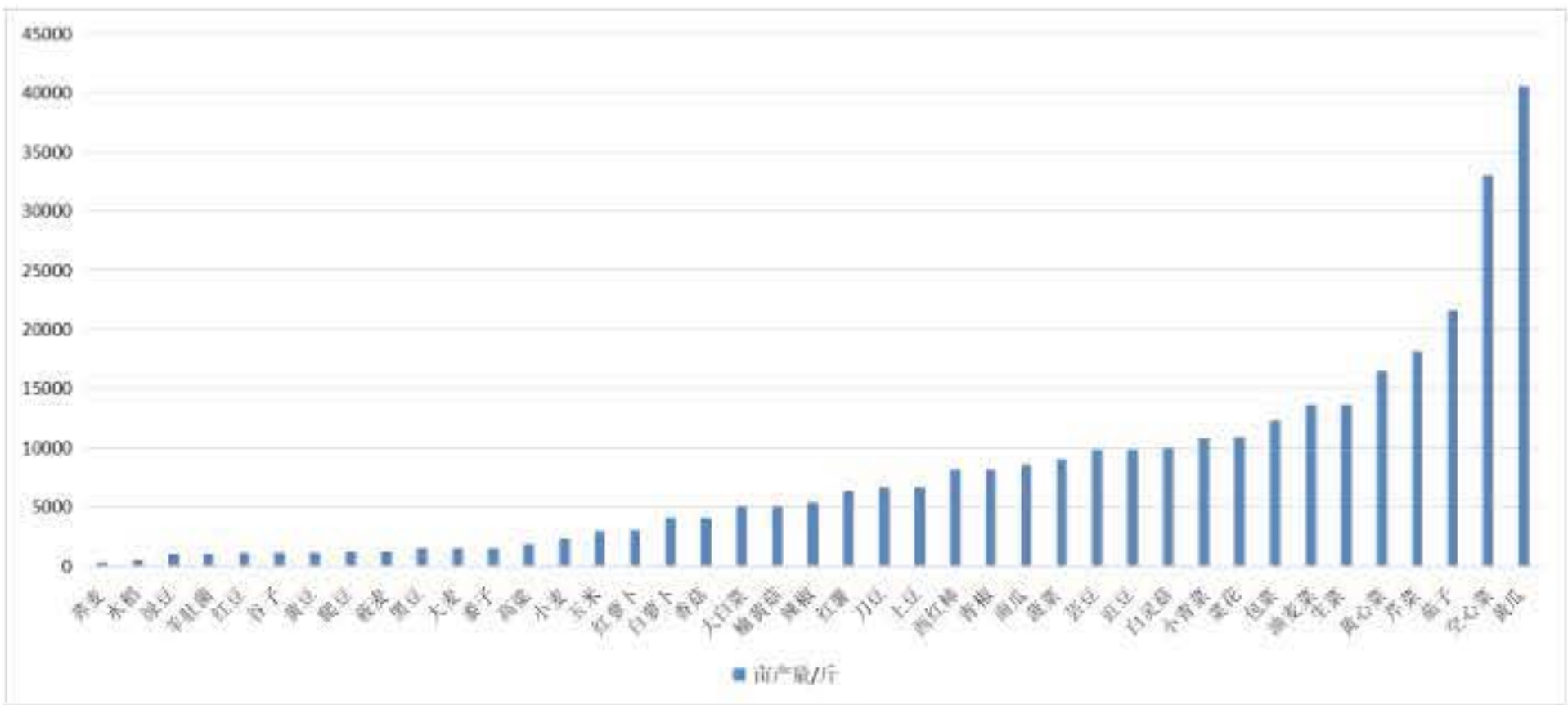


图 3 2023 年不同农作物总亩产量

其次，对 2023 年不同农作物总亩产量进行可视化，结果如图 3 所示。由图 3 可以看出，种植黄瓜的亩产量高达 40500 斤，分析数据可知在普通大棚、智慧大棚、水浇地上黄瓜亩产量分别达到 15000 斤、13500 斤、12000 斤。大棚有良好的保温和遮阳条件，适宜黄瓜生长，故亩产量高。种植在智慧大棚的面积小于普通大棚，故智慧

大棚的亩产量略小于普通大棚亩产量。

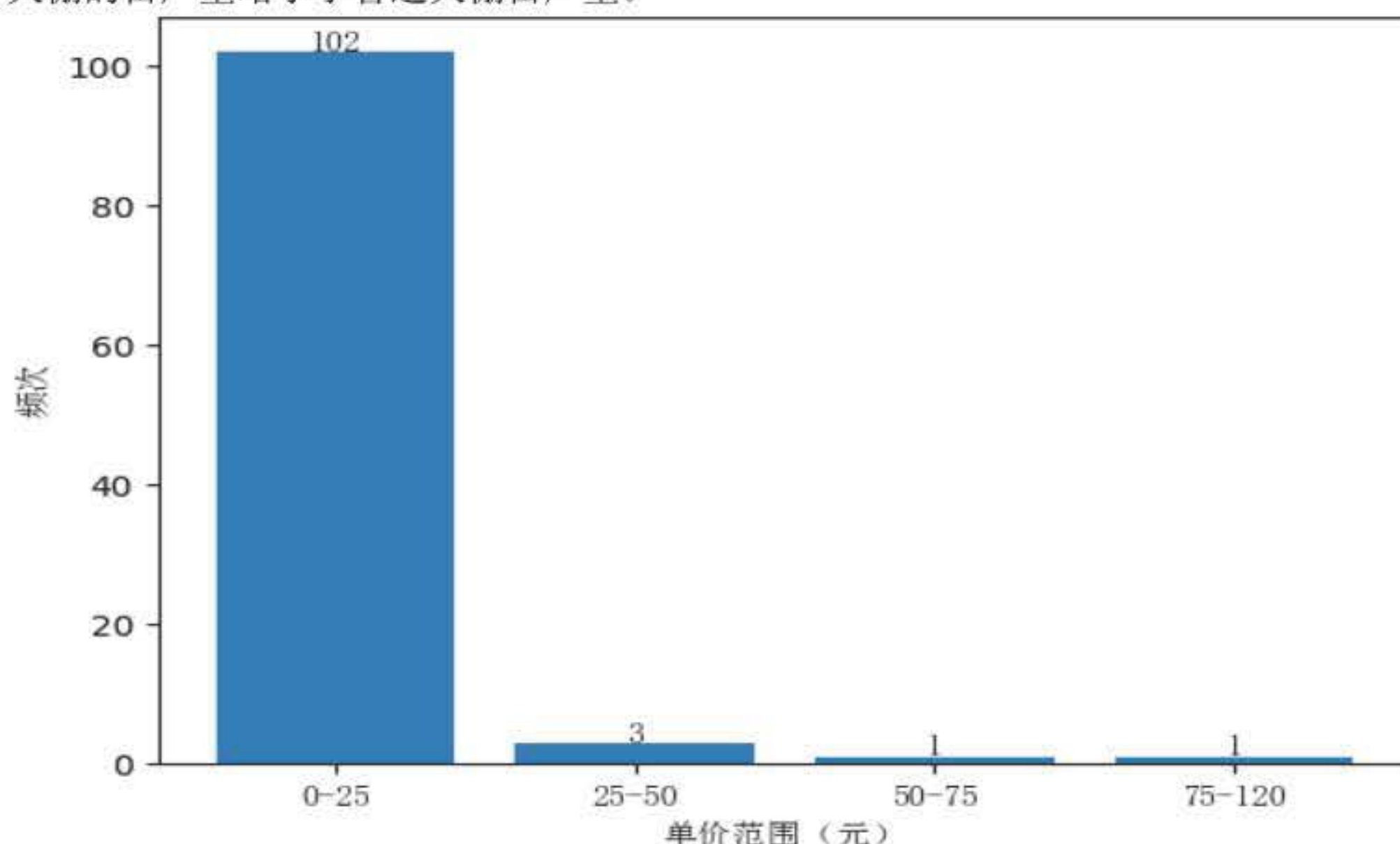


图 4 2023 年不同地块不同农作物的销售单价范围分布

第三，对 2023 年种植在不同地块的不同农作物的销售单价范围进行可视化，结果如图 4 所示。可以看出大多数农作物单价范围在 0-25 元，少部分分布在更大的范围内，可能与不同环境下种植成本与亩产量的差异有关。

最后，由于各种农作物的各项预期数据相对于 2023 年保持稳定，且每季种植的农作物均在当季销售。我们近似认为，未来七年各种农作物的种植成本、亩产量与 2023 年的对应数据相等；此外，我们认为 2023 年农作物销售市场达到供需平衡，故预期销售量用 2023 年每季度农作物的总产量来代替；每种农作物的种植成本和亩产量会受到种植地块环境的影响，销售单价在每季具有较大变化，为简化模型，我们取区间均值作为预期销售单价。

综合上述分析，得到不同地块不同作物的属性信息表，在附件：地块作物属性表.xlsx 中展示。对于表中变量有如下定义： i 表示地块的编号，从 A1 到 F4 分别编号为 1~54， j 表示每种农作物的编号（编号为确定值）， X_{ij} 表示在第 i 块地上种植编号为 j 的农作物，由于每种地块并非适宜种植所有农作物，表格中 j 仅显示适宜种植的农作物编号。另外，考虑到每季种植的农作物在当季销售，故对于适宜种植两季作物的地块，上表给出此类地块每季适宜种植的农作物相关属性，故我们可以分别计算两季的收入，进而求和得到总收入。

5.2 构建非线性规划模型

该乡村共有四种耕地和两种大棚，考虑到农作物生长规律以及地块特点，我们首先筛选出每种地块在每季适宜种植的农作物类型，再为筛选出的农作物分配耕地以达到最大收益。

对于单季作物，直接计算其一年内的利润；对于两季作物，分析表格可知两季作物基本只能在一年中种植一季，因此我们分别计算第一季、第二季作物的收益，进而

求和得到这一年该地块的总收益。

以 2024 年作为第一年，我们定义 X_{ij}^k 为第 k 年中，第 i 种地块上第 j 种作物的种植面积； N_{ij} 表示第 j 种作物在第 i 种地块上的亩产量； P_j 表示第 j 种作物的销售单价； C_{ij} 表示第 j 种作物在第 i 种地块上的种植成本； Y_j 表示第 j 种作物的预期总销量； X_i 表示第 i 种地块的总面积； W_i^k 表示在第 k 年第 i 种地块的种植总收益， W 表示 2024~2030 年的种植总收益。（其中 k 取 1~7，分别表示 2024~2030 年； i 取 1~54，分别表示地块 A1~F4； j 取 1~41，分别表示黄豆~羊肚菌）

由于不能连续种植同一种作物，因此可能在第一年达到最大收益时第二年无法达到最大收益。为实现可持续的种植，获得尽可能大的收益并降低滞销浪费，我们将七年的总收益 W 作为判断标准，针对以下两种情况，分别设置不同的目标函数及约束条件。

5.2.1 超产部分滞销，造成浪费

目标函数如下：

$$\begin{aligned} \max W &= \sum_k \sum_i W_i^k \\ &= \sum_{k=1}^7 \sum_{i=1}^{54} \sum_{j=1}^{41} (X_{ij}^k \times N_{ij} \times P_j - C_{ij} \times X_{ij}^k) \end{aligned} \quad (1)$$

有以下约束条件：

1、对于单季作物，每种作物在同一地块（含大棚）都不能连续重茬种植，否则会减产；对于两季作物，考虑到种植时间的间隔性，故连续两年的第一季或第二季均可种植同一作物，但是一年之内的两季不能种植同一作物。

$$X_{ij}^k \cdot X_{ij}^{k+1} = 0 \quad (2)$$

其中 j 取单季作物， $j=1, 2, 3, \dots, 16$ ；

$$X_{ij}^{k_1} \cdot X_{ij}^{k_2} = 0 \quad (3)$$

其中 j 取两季作物， $j=17, 18, \dots, 41$ ； k_1 为第 k 年第一季种植， k_2 为第 k 年第二季种植。

2、因含有豆类作物根菌的土壤有利于其他作物生长，从 2023 年开始要求每个地块（含大棚）的所有土地三年内至少种植一次豆类作物。

$$X_{ij}^k + X_{ij}^{k+1} + X_{ij}^{k+2} > 0 \quad (4)$$

其中 j 取豆类作物的序号， $j=1, 2, 3, 4, 5, 17, 18, 19$ 。

3、种植必须满足预期销售量的标准。

$$\sum_i X_{ij}^k \times N_{ij} \geq Y_j \quad (5)$$

4、对于单季作物，第 i 块地的种植总面积应该小于等于该地总面积；对于两季作

物，第 i 块地每一季种植总面积应小于等于该地总面积。

$$\sum_j X_{ij}^k \leq X_i \quad (6)$$

其中 j 取 1~16 为所有单季作物；取 17~34 为所有适宜第一季种植的作物；取 17~41 中所有适宜第二季种植的作物。

5、对于某地块的所有适宜种植的作物类型，需满足以下条件：要么不种植，要么种植面积不得小于该地块（含大棚）总面积的 δ 倍。其中 δ 为可变系数，根据不同地块的实际情况取不同值。

$$\delta X_i \leq X_{ij}^k \leq X_i \text{ 或 } X_{ij}^k = 0 \quad (7)$$

综合上述条件，结果如下：

$$\max W = \sum_k \sum_i W_i^k = \sum_k \sum_i \sum_j (X_{ij}^k \times N_{ij} \times P_j - C_{ij} \times X_{ij}^k) \quad (8)$$

$$s.t. \begin{cases} X_{ij}^k \cdot X_{ij}^{k+1} = 0 \\ X_{ij}^{k_1} \times X_{ij}^{k_2} = 0 \\ X_{ij}^k + X_{ij}^{k+1} + X_{ij}^{k+2} > 0 \\ \sum_i X_{ij}^k \times N_{ij} \geq Y_j \\ \sum_{j=1}^{16} X_{ij}^k \leq X_i \\ \sum_{j=17}^{34} X_{ij}^{k_1} \leq X_i \\ \sum_{j=17}^{41} X_{ij}^{k_2} \leq X_i \\ \delta X_i \leq X_{ij}^k \leq X_i \text{ 或 } X_{ij}^k = 0 \end{cases} \quad (9)$$

5.2.2 超产部分半价出售

在第一小问的基础上，考虑到超产部分进行半价出售，修改目标函数；约束条件删去第一小问的约束条件 3 即可，最终结果如下所示：

$$\begin{aligned} \max W &= \sum_k \sum_i W_i^k \\ &= \sum_k \sum_i \sum_j \left[(X_{ij}^k \times N_{ij} - Y_j) \times \frac{1}{2} P_j + Y_j \times P_j - C_{ij} \times X_{ij}^k \right] \end{aligned} \quad (10)$$

$$s.t. \begin{cases} X_{ij}^k \cdot X_{ij}^{k+1} = 0 \\ X_{ij}^{k_1} \times X_{ij}^{k_2} = 0 \\ X_{ij}^k + X_{ij}^{k+1} + X_{ij}^{k+2} > 0 \\ \sum_{j=1}^{16} X_{ij}^k \leq X_i \\ \sum_{j=17}^{34} X_{ij}^k \leq X_i \\ \sum_{j=17}^{41} X_{ij}^k \leq X_i \\ \delta X_i \leq X_{ij}^k \leq X_i \text{ 或 } X_{ij}^k = 0 \end{cases} \quad (11)$$

5.3 模型求解

这是一个非线性规划问题，我们引入二进制变量 Z_{ij}^k （取值为 0 或 1），表示在第 k 年某块地上是否种植作物 j 。在定义决策变量时，需添加该二进制变量的定义以控制变量 X 。例如某年某块地上种植了某种作物，即 $Z_{ij}^k = 1$ ，在此前提下，求解 X_{ij}^k 的具体值。此外，我们可以在约束条件中引入该二进制变量，简化约束条件以降低求解难度。综合以上分析，我们使用 python 的 pulp 库，构建目标函数和约束条件，得到最优解。

5.3.1 超产部分滞销，造成浪费

结果如 result1_1.xlsx 所示，我们计算出每块地中种植的各种作物面积，进而得到最大利润：34623417.97。种植面积随年份、地块、作物类型变化的三维图如下所示。

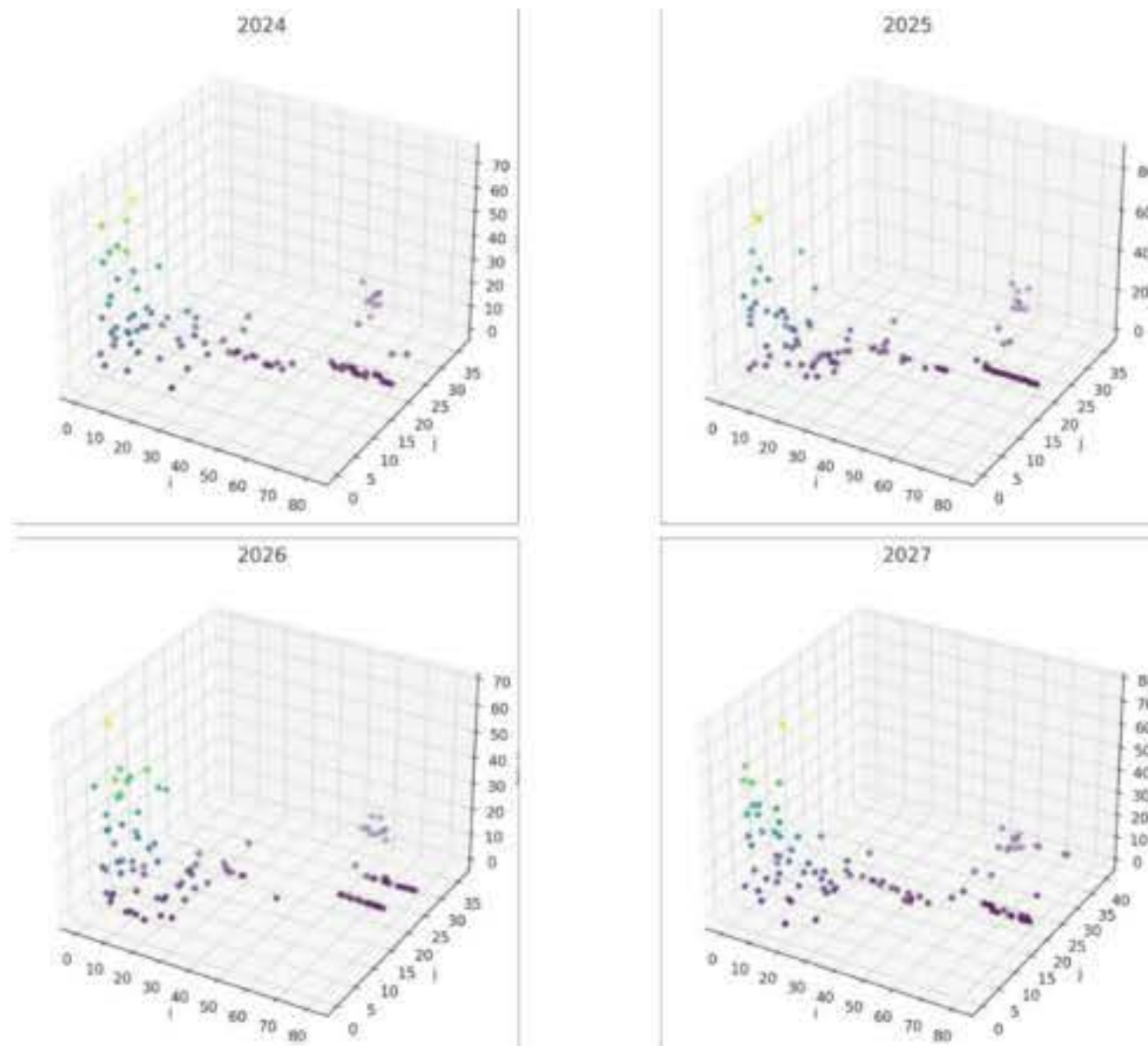


图 5 种植面积随年份、地块、作物类型变化的三维图 1

5.3.2 超产部分半价出售

结果如 result1_2.xlsx 所示，我们计算出每块地中种植的各种作物面积，进而得到最大利润：85817862.12。种植面积随年份、地块、作物类型变化的三维图如下所示。

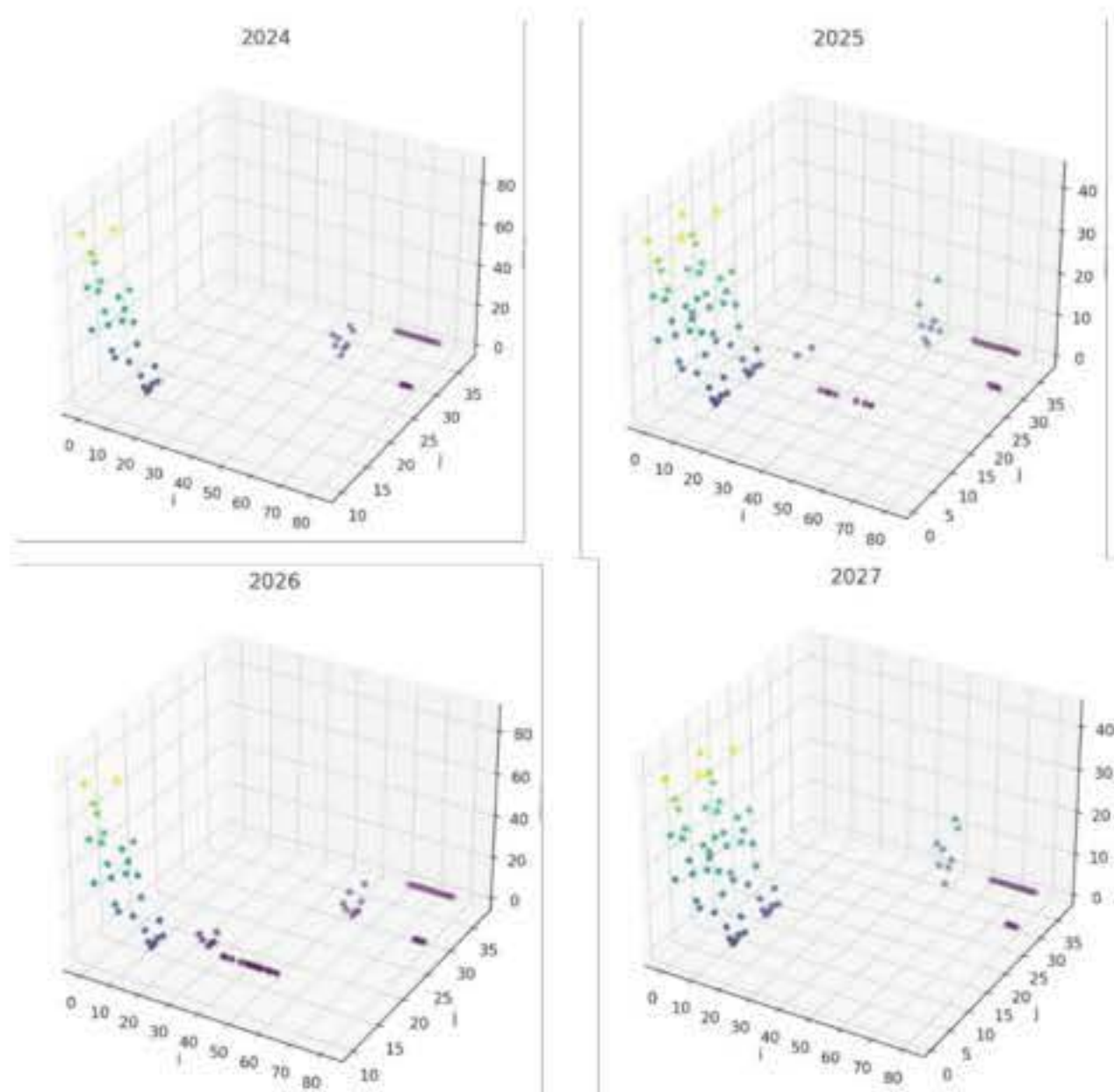


图 6 种植面积随年份、地块、作物类型变化的三维图 2

5.4 结论分析

综合以上分析，我们可以得到如下结论：超产按照半价出售能获得更大利润，超产部分滞销处理会造成较大程度的浪费。但两种情况也具有一定的共性，即在地块面积、农作物生长规律及地块环境等约束下，我们应当多种植小麦、玉米等预期销售量较高，成本较低的作物，在大棚中我们应种植一些两季作物，因地制宜优化种植策略，满足市场需求的同时实现较高的收益。

5.5 模型的优化调整

优化一：在上述模型的求解过程当中，设置了若干条件以简化模型。首先我们将各种农作物未来的预期销售量、种植成本、亩产量和销售价格相对于 2023 年保持稳定这一信息简化为保持不变，实际上保持稳定允许出现微小的波动，为优化模型，我们设定其变化的幅度为 $\pm 1\%$ 。

优化二：我们认为 2023 年农作物销售市场达到供需平衡，预期销售量用 2023 年每季度农作物的总产量来等价。但实际上作物的市场情况应当分为三类考虑：供等于需、供大于需、供小于需。我们可结合具体市场行情，对每种作物的情况分类，以优化对预期销售量的模拟。

优化三：作物的销售单价处于一定的区间，上述将预期销售单价简化为 2023 年作物单价区间的均值，这实际上是不准确的。考虑到作物的价格受季节影响较大，针对不同季节我们可以得到不同价格，且这一规律逐年保持稳定。

优化四：考虑到有限的管理与种植技术，作物的种植分布不应太分散。我们可以

增加一个集中约束，每种地的多个地块通常分布在同一区域，可约束每种作物最多只能种在两种地形。

通过以上分析，优化模型如下所示：

5.5.1 目标函数

考虑到销售价格波动和供需关系，目标是最大化 2024~2030 年期间的总收益。

$$\max W = \sum_{k=1}^7 W_k \quad (12)$$

其中 W_k 表示第 k 年的收益，计算公式如下：

$$W_k = \sum_{i=1}^{54} \sum_{j=1}^{41} X_{ij}^k \times N_{ij} \times P_j^k \times S_{ij} - \sum_{i=1}^{54} \sum_{j=1}^{41} X_{ij}^k \times C_{ij} \quad (13)$$

5.5.2 约束条件

添加作物集中种植约束进行优化，即每种作物至多只能种植在两种地形上。

$$\sum_j Z_{ij}^k \leq 2 \quad (14)$$

5.5.3 作物价格波动

作物的销售单价考虑季节性和波动性，以区间表示，设定为：

$$P_j^k = P_j \times (1 + \mu \pm 0.01) \quad (15)$$

其中， P_j 是 2023 年基准价格， μ 为季节性因素系数，通过历史数据拟合。

5.5.4 模型求解

结合新的约束条件，该优化模型可通过线性规划或非线性规划，使用 Python 中的 PuLP 或 Gurobi 等优化库进行求解。

受到市场供需，季节变化，微小波动的影响，结果在一定小范围内动态变化，可由此制定出更加稳定的种植策略。

六、问题二模型的建立与求解

6.1 模型的建立

在问题二中，亩产量、销售单价、种植成本、预期销售量会随年份呈现不同的变化，并且还要考虑潜在的风险，即这是一个具有随机变量的最优种植方案求解问题。

6.1.1 目标函数

为实现收益最大化，建立如下目标函数：

$$\max W = \sum_{k=1}^7 \sum_{i=1}^{54} \sum_{j=1}^{41} [(X_{ij}^k N_{ij}^k P_j^k - C_{ij}^k X_{ij}^k) (1 - r_l^k)] \quad (16)$$

其中 W 表示从 2024~2030 年的总收益；我们定义 X_{ij}^k 为第 k 年中，第 i 种地块上第 j 种作物的种植面积； N_{ij}^k 表示第 k 年中，第 j 种作物在第 i 种地块上的亩产量； P_j^k 表示第 k 年中，第 j 种作物的销售单价； C_{ij}^k 表示第 k 年中，第 j 种作物在第 i 种地块上的

种植成本； Y_j^k 表示第 k 年中，第 j 种作物的预期总销量； X_i 表示第 i 种地块的总面积； r_i^k 表示风险调整系数。

6.1.2 约束条件

1、对于单季作物，每种作物在同一地块（含大棚）都不能连续重茬种植，否则会减产；对于两季作物，考虑到种植时间的间隔性，故连续两年的第一季或第二季均可种植同一作物，但是一年之内的两季不能种植同一作物。

$$X_{ij}^k \cdot X_{ij}^{k+1} = 0 \quad (17)$$

其中 j 取单季作物， $j=1, 2, 3, \dots, 16$ ；

$$X_{ij}^{k_1} \cdot X_{ij}^{k_2} = 0 \quad (18)$$

其中 j 取两季作物， $j=17, 18, \dots, 41$ ； k_1 为第 k 年第一季种植， k_2 为第 k 年第二季种植。

2、因含有豆类作物根菌的土壤有利于其他作物生长，从 2023 年开始要求每个地块（含大棚）的所有土地三年内至少种植一次豆类作物。

$$X_{ij}^k + X_{ij}^{k+1} + X_{ij}^{k+2} > 0 \quad (19)$$

其中 j 取豆类作物的序号， $j=1, 2, 3, 4, 5, 17, 18, 19$ 。

3、种植必须满足预期销售量的标准。

$$\sum_i X_{ij}^k \times N_{ij} \geq Y_j \quad (20)$$

4、对于单季作物，第 i 块地的种植总面积应该小于等于该地总面积；对于两季作物，第 i 块地每一季种植总面积应小于等于该地总面积。

$$\sum_j X_{ij}^k \leq X_i \quad (21)$$

其中 j 取 1~16 为所有单季作物；取 17~34 为所有适宜第一季种植的作物；取 17~41 中所有适宜第二季种植的作物。

5、对于某地块的所有适宜种植的作物类型，需满足以下条件：要么不种植，要么种植面积不得小于该地块（含大棚）总面积的 δ 倍。其中 δ 为可变系数，根据不同地块的实际情况取不同值。

$$\delta X_i \leq X_{ij}^k \leq X_i \text{ 或 } X_{ij}^k = 0 \quad (22)$$

综合以上条件得到：

$$\max W = \sum_{k=2024} \sum_{i=1} \sum_{j=1} [(X_{ij}^k N_{ij}^k P_j^k - C_{ij}^k X_{ij}^k) (1 - r_i^k)] \quad (23)$$

$$s.t. \begin{cases} X_{ij}^k \cdot X_{ij}^{k+1} = 0 \\ X_{ij}^{k_1} \times X_{ij}^{k_2} = 0 \\ X_{ij}^k + X_{ij}^{k+1} + X_{ij}^{k+2} > 0 \\ \sum_i X_{ij}^k \times N_{ij} \geq Y_j \\ \sum_{j=1}^{16} X_{ij}^k \leq X_i \\ \sum_{j=17}^{34} X_{ij}^k \leq X_i \\ \sum_{j=17}^{41} X_{ij}^k \leq X_i \\ \delta X_i \leq X_{ij}^k \leq X_i \text{ 或 } X_{ij}^k = 0 \end{cases} \quad (24)$$

6.1.3 不确定性和风险调整

未来预期销售量、亩产量、种植成本、销售价格都是随年份波动的随机变量，假设变量服从下列分布，依据其分布进行模拟求解。

1、亩产量 N_{ij}^k ：

综合题目信息及相关文献[3]，农作物的亩产量增长率认为符合伯努利分布 $20\%B(1,0.5)-10\%$ 。

$$N_{ij}^k = N_{ij}^{2023} (1 + \delta_j), \delta_j \sim 20\%B(1, 0.5) - 10\% \quad (25)$$

2、预期销售量 Y_j^k ：

对于小麦和玉米 ($j=6, 7$)，认为预期年销售量增长率符合均匀分布 $U(5\%, 10\%)$ ，对于其他农作物的预期销售量增长率，认为符合伯努利分布 $10\%B(1,0.5)-5\%$ 。

$$Y_j^k = Y_j^{2023} (1 + \sigma_j)$$

$$\sigma_j \sim U(5\%, 10\%), j = 6, 7 \quad (26)$$

$$\sigma_j \sim 10\%B(1, 0.5) - 5\%, j \neq 6, 7$$

3、种植成本 C_{ij}^k ：

农作物的种植成本增长率认为符合正态分布 $N(5\%, 1\%)$ ，满足题目信息使增长率在 5% 左右进行小幅度变化。

$$C_{ij}^k = C_{ij}^{2023} (1 + \theta_j), \theta_j \sim N(5\%, 1\%) \quad (27)$$

4、销售价格 P_j^k ：

蔬菜类作物价格增长率认为符合正态分布 $N(5\%, 1\%)$ ， $j=1 \sim 16$ ；食用菌价格增长率认为符合均匀分布 $U(-5\%, -1\%)$ ， $j=38 \sim 49$ ；羊肚菌销售单价增长率认为符合正态分布 $N(-5\%, 1\%)$ ， $j=41$ ；为简化模型，假设粮食类作物销售价格保持不变。

r_l^k 表示第 k 年第 l 种类型地块 (l 取 1~6) 的风险调整系数, r_l^k 取值在 0~1 之间, 风险与取值大小呈正相关。查找文献可知^[4], 如果发生干旱, 则对于梯田和水浇地来说影响较小, 风险调整系数较小, 其他影响则较大; 如果发生沙尘暴, 则对于山坡地来说影响较小, 风险调整系数较小, 其他影响则较大; 如果发生暴雪、暴雨, 则所有类型的地块风险调整系数都较大。假设每年发生极端情况的概率为 1%, 且四种极端情况发生的概率相等。在此基础上, 通过随机模拟每年的极端天气和风险调整系数进行问题求解, 且风险调整系数最大不超过 0.3。

$$P_j^k = P_j^{2023} (1 + \varepsilon_j)$$

$$\begin{cases} \varepsilon_j \sim N(5\%, 1\%), j = 1 \sim 16; \\ \varepsilon_j \sim U(-5\%, -1\%), j = 38 \sim 49; \\ \varepsilon_j \sim N(-5\%, 1\%), j = 41; \\ \varepsilon_j = 0, j \text{ 取其他值;} \end{cases} \quad (28)$$

6.2 模型求解

这是一个涉及随机波动变量的复杂优化问题, 难以使用确定性的方法解决, 故本题利用蒙特卡洛模拟求解最优种植方案。

蒙特卡洛模拟是一种基于随机抽样的计算方法, 用于估计复杂系统的行为或求解难以解析计算的问题。其核心思想是通过大量的随机试验, 模拟现实中不确定的现象或过程, 逐渐收敛到期望的结果。解决本问题的基本思路是利用蒙特卡洛方法随机模拟变量, 生成多个未来年份的收益分布, 最后通过统计分析选取收益最大的种植方案。

Step1 生成随机样本:

使用 Python 语言生成大量随机样本。每个随机变量的样本数为 1000, 以确保模拟的有效性和可靠性。

Step2 蒙特卡洛模拟:

通过抽样随机波动变量的分布, 生成大量可能的未来情景。对每个情景, 执行以下操作:

1、计算每种作物的预期收益: 基于随机生成的预期销售量、亩产量、种植成本和销售单价。

2、求解规划问题: 在每个抽样情景下求解优化问题, 最大化总收益。

Step3 结果分析:

收集数据: 对所有模拟的收益进行统计分析。

计算关键指标: 找出最大收益的平均值、最大值、标准差等。

决策建议: 基于模拟结果, 推荐能在大多数情况下保持最大总收益的策略。

Step4 风险评估与决策优化:

风险评估: 分析不同情景下收益的波动性, 确定哪些变量最影响收益的稳定性。

敏感性分析: 评估关键随机变量的变化如何影响最终收益, 从而调整策略以应对极端情况。

6.3 结论分析

6.3.1 不同情景下最大收益分布情况

我们得到不同情景下最大收益分布情况, 并进行可视化。

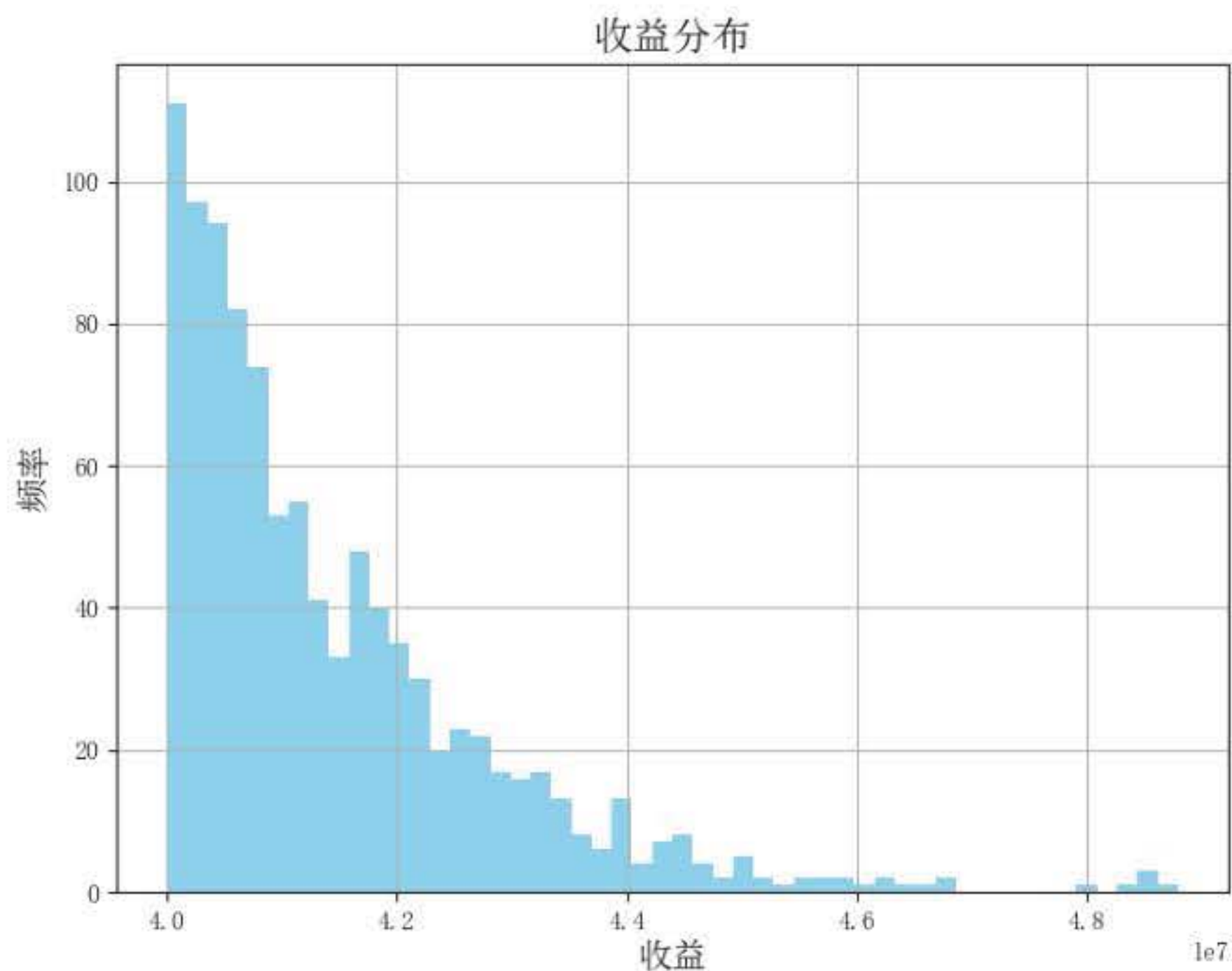


图 7 不同情景下最大收益分布图

由图可知，最大收益均在 4000 万~4900 万之间，多数分布在 4000 万~4400 万之间。

表 1 样本最大收益值分析表

样本中最大收益值最大值	48786287.63862544
样本中最大收益值平均值	41443256.61561046
样本中最大收益值标准差	1381606.055269333
样本中最大收益值最小值	40102122.969087206

我们分析模拟结果，发现有一组策略在所有样本中应用时较为接近最大收益平均值，将结果记录进 result2.xlsx 中。

6.3.2 最优种植策略收益波动情况

最优种植策略应用在不同样本时，收益波动情况如下图所示，发现收益集中在 4000 万~4400 万之间波动。波动相对规律稳定，表明外部环境（如市场价格、作物产量等）对收益的影响不大，说明该最优种植方案在现实中具有较好的适应性和抗风险

能力。

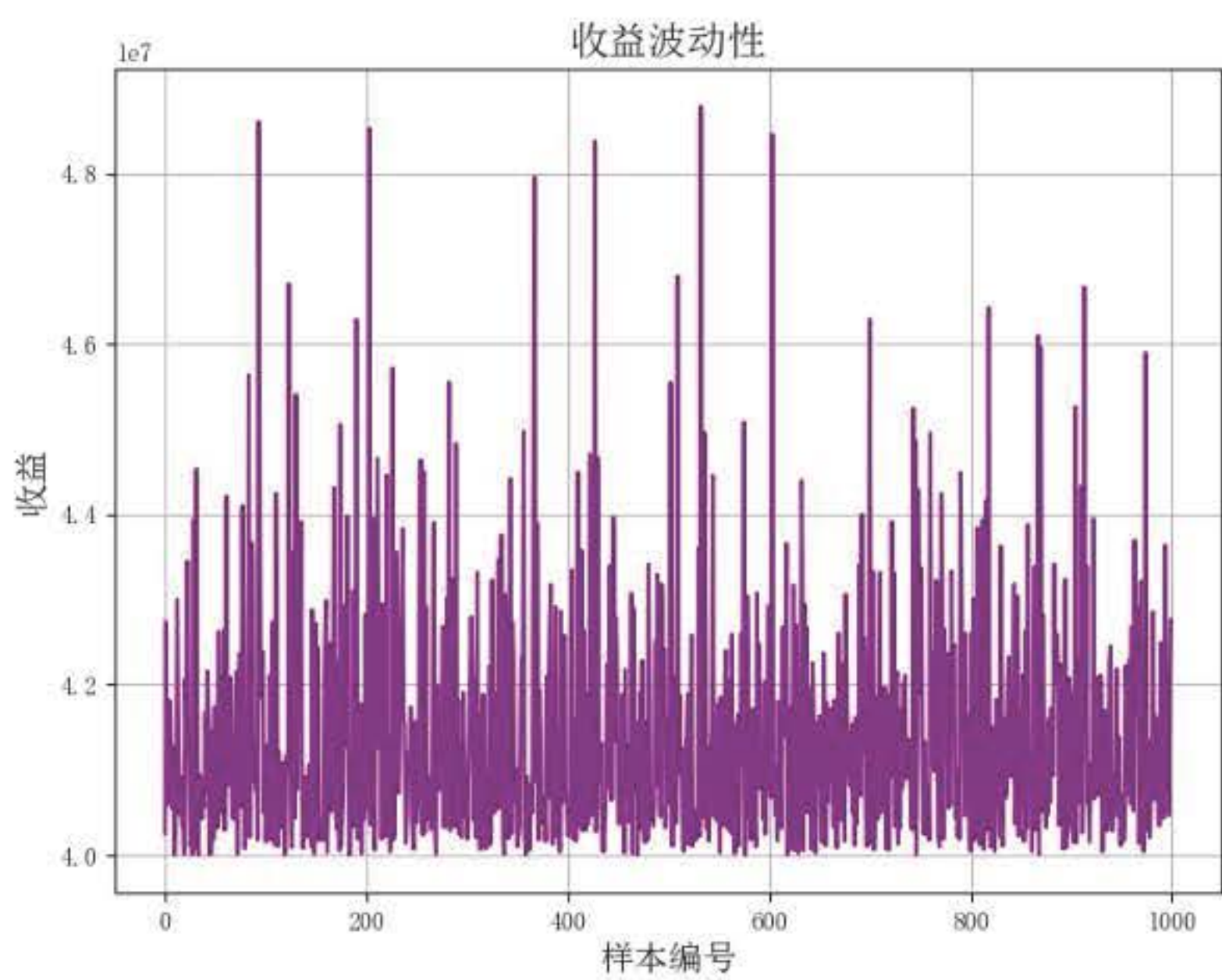


图 8 最优种植策略收益波动情况图

6.3.3 收益与随机变量的相关性以及影响程度

收益与随机变量的相关性情况如下图所示。销售单价、预期销售量、亩产量与收益呈现较强的正相关，种植成本与收益呈现较弱的负相关。四种变量中，亩产量的影响最大。在计算最优种植方案时，应着重考虑亩产量，以使方案有较强的稳定性。

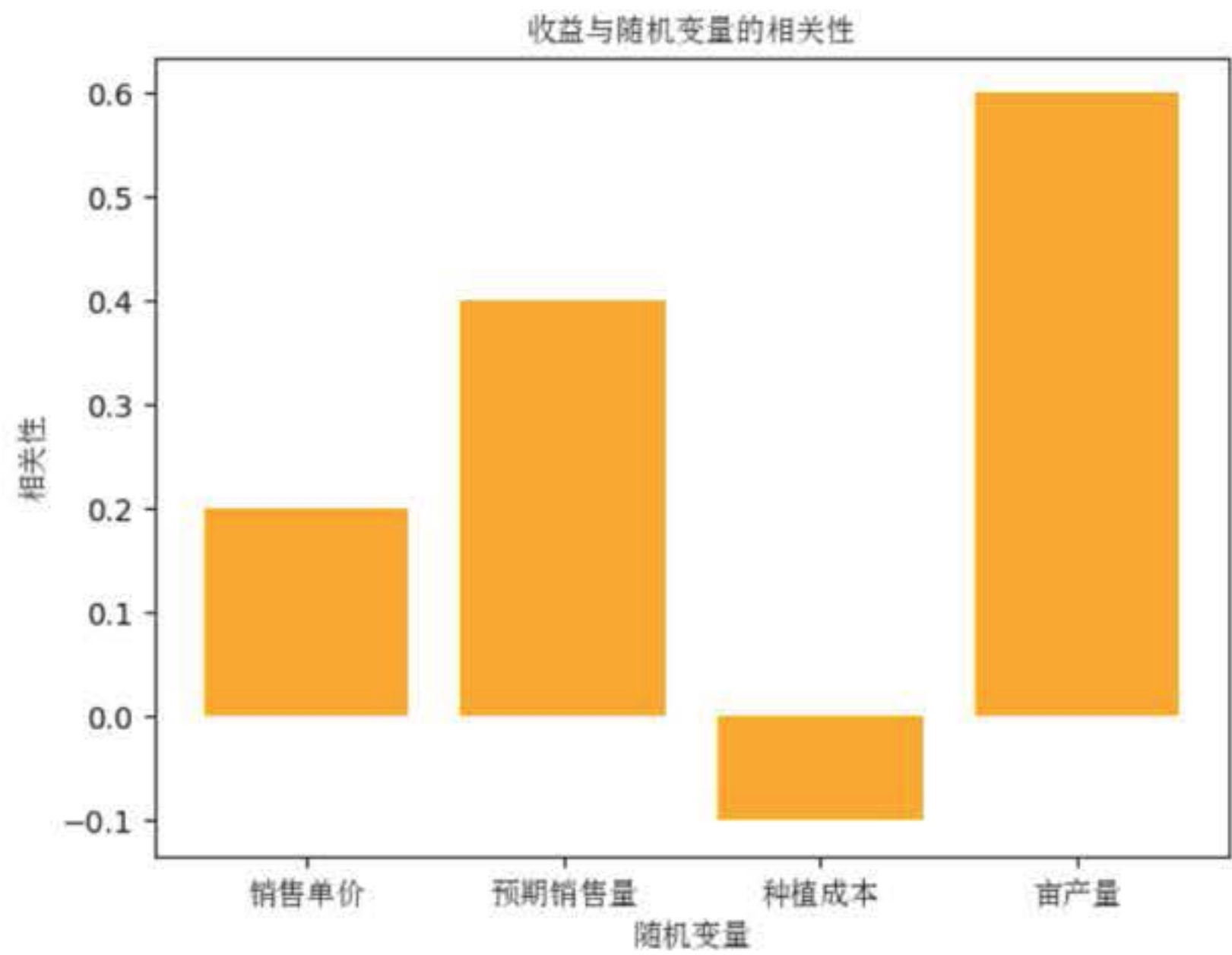


图 9 收益与随机变量的相关性图

七、问题三模型的建立与求解

7.1 相关性分析

引入皮尔逊相关系数，将问题三简化为求解预期销售量、销售价格、种植成本三个变量之间的相互关系。

若两组数据 $X: \{X_1, X_2, \dots, X_n\}$ 和 $Y: \{Y_1, Y_2, \dots, Y_n\}$ 是样本数据，分别代表预期销售量、销售价格、种植成本。则有：

$$\bar{X} = \frac{\sum_{i=1}^n X_i}{n}, \bar{Y} = \frac{\sum_{i=1}^n Y_i}{n} \quad (29)$$

$$Cov(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{n-1} \quad (30)$$

$$r_{xy} = \frac{Cov(X, Y)}{S_X S_Y} \quad (31)$$

$$S_X = \sqrt{\frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n-1}}, S_Y = \sqrt{\frac{\sum_{i=1}^n (Y_i - \bar{Y})^2}{n-1}} \quad (32)$$

其中 $\bar{X}, \bar{Y}$ 为样本均值， $Cov(X, Y)$ 为样本协方差， r_{xy} 为样本 pearson 相关系数， S_X, S_Y 分别为数据 X 和 Y 的标准差。利用 python 求解得到皮尔逊相关系数热力图。

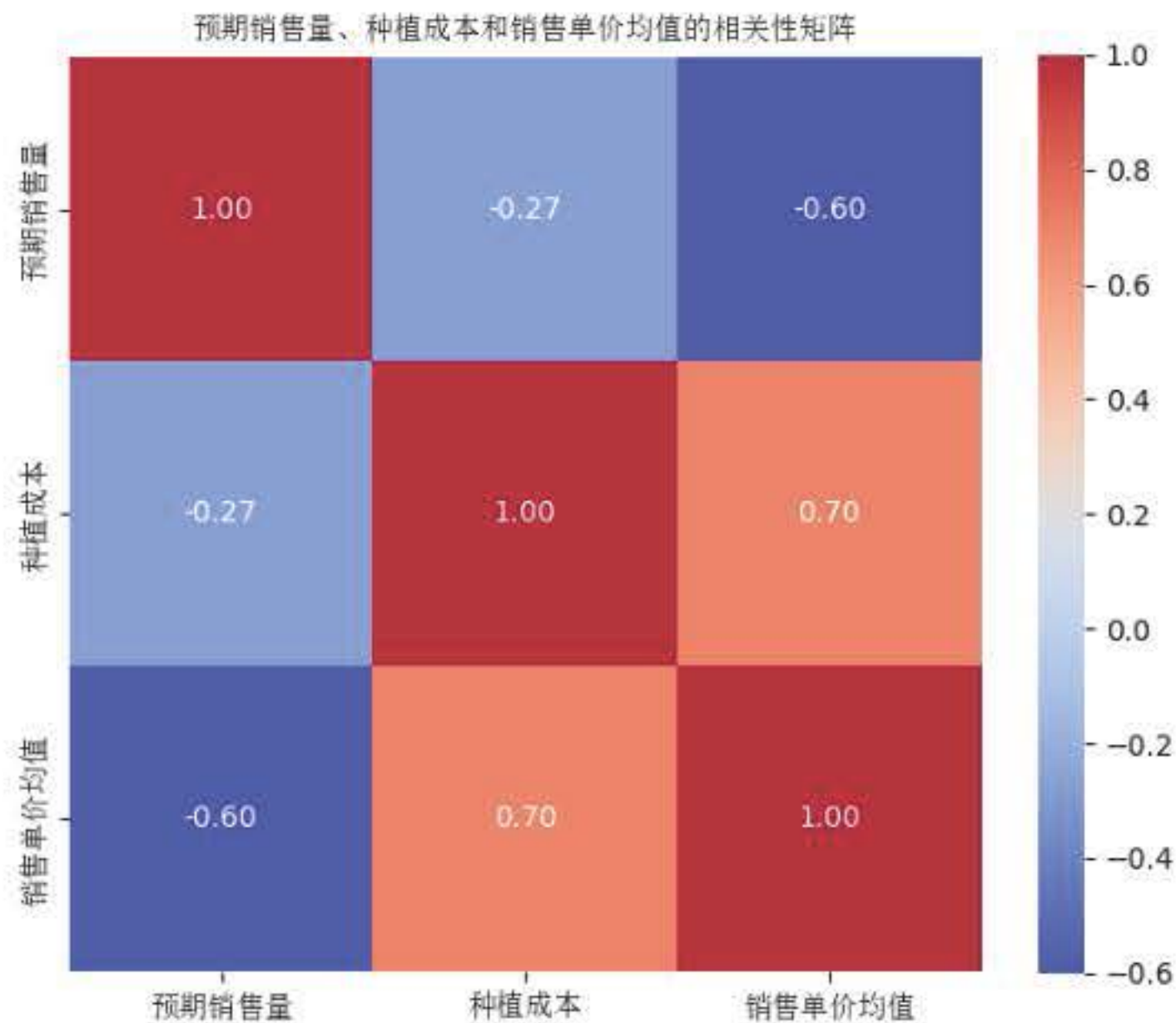


图 10 皮尔逊相关系数热力图

分析上图，发现销售单价均值和种植成本的皮尔逊相关系数为 0.7，具有较强相关性；而种植成本和预期销售量相关性较弱；预期销售量和销售单价均值的皮尔逊相关系数为-0.6，具有较强的负相关。考虑现实情况，为保证收益，种植成本增加时，销售单价会随之增加一定幅度，因此呈现较强正相关；当销售单价均值较大时，预期销售量相对减小，因此呈现较强负相关。

7.2 可替代性和互补性分析

农作物的可替代性指两种或多种作物在生产或消费中能够相互替代的程度。种植者可以根据市场需求、气候条件或利润率来选择种植哪种作物。如果某种作物的价格或收益下降，种植其他作物或者提高其他作物的价格同样能满足市场需求并获得较高利润。

为了反映不同农作物间的替代性，我们可以假设某些作物会影响其他作物的市场价格。通过引入一个替代效应修正因子 θ_{jm} ，这个因子表示种植作物 j 的总利润贡献率下降（即产量或单价减少）时，作物 m 的贡献率将会相应上升，反映了作物间的替代性。 θ_{jm} 为作物 j 对作物 m 价格的替代效应修正因子。

$$\max W = \sum_{ijk} [(X_{ij}^k N_{ij}^k P_j^k \theta_{jm} - C_{ij}^k X_{ij}^k) (1 - r_l^k)] \quad (33)$$

农作物的互补性通常指某些作物的种植会有助于其他作物的生长，或者是作物之间的产品能够一起使用以增加市场价值。例如，土壤在种植豆类作物之后有利于其他作物的生长。

为研究作物间的互补性，我们可以在成本项中引入一个互补性修正因子 φ_{jm} ，表示作物 j 和作物 m 之间的互补性对成本的影响。如果 j 和 m 具有互补性，则 φ_{jm} 小于 1，表示作物 m 帮助降低作物 j 的种植成本。

$$\max W = \sum_{ijk} [(X_{ij}^k N_{ij}^k P_j^k \theta_{jm} - \varphi_{jm} C_{ij}^k X_{ij}^k) (1 - r_l^k)] \quad (34)$$

7.3 构建优化模型

基于问题二的非线性规划模型，考虑到农作物之间的替代性和互补性，确定新的目标函数及约束条件。

目标函数如下所示， θ_{jm} 为替代效应修正因子， φ_{jm} 为互补性修正因子：

$$\max W = \sum_{ijk} [(X_{ij}^k N_{ij}^k P_j^k \theta_{jm} - \varphi_{jm} C_{ij}^k X_{ij}^k) (1 - r_l^k)] \quad (35)$$

约束条件在问题二的基础上，进行补充：

$$s.t. \left\{ \begin{array}{l} X_{ij}^k \cdot X_{ij}^{k+1} = 0 \\ X_{ij}^{k_1} \times X_{ij}^{k_2} = 0 \\ X_{ij}^k + X_{ij}^{k+1} + X_{ij}^{k+2} > 0 \\ \sum_i X_{ij}^k \times N_{ij} \geq Y_j \\ \sum_{j=1}^{16} X_{ij}^k \leq X_i \\ \sum_{j=17}^{34} X_{ij}^k \leq X_i \\ \sum_{j=17}^{41} X_{ij}^k \leq X_i \\ \delta X_i \leq X_{ij}^k \leq X_i \text{ 或 } X_{ij}^k = 0 \end{array} \right. \quad (36)$$

本问题进一步优化问题二的非线性规划模型，使用蒙特卡洛方法进行求解。已知预期销售量和销售单价均值成负相关，销售单价均值和种植成本成正相关，故模拟分布时，当模拟出的预期销售量增长率较小时，销售单价增长率也较小，反之亦然；同样当模拟出的中种植成本增长率较小时，销售单价增长率也较小，反之亦然。

7.4 结果对比

我们分析模拟结果，发现了一组策略在应用于所有样本时较为接近最大收益平均值。我们将所有的样本都应用该策略得到了下图所示的收益波动图。

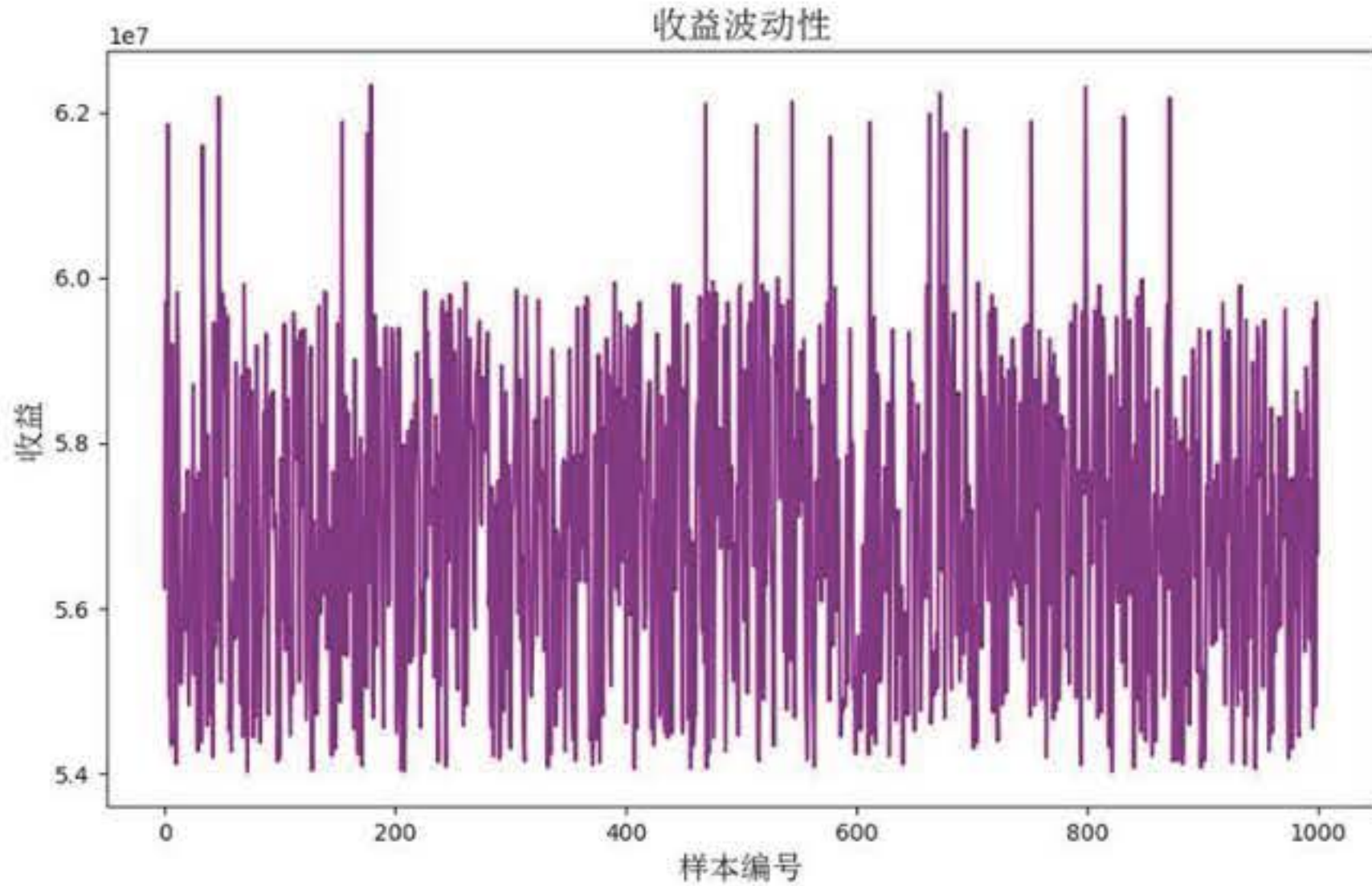


图 11 最优种植策略收益波动情况图

由上图我们可以分析得到以下结论:应用问题三模型后,所有样本的收益都得到了大幅提升,并且样本之间的波动性较小,这说明我们的模型在考虑到预期销售量与销售价格、种植成本之间的相关性以及作物之间的替代性互补性之后能够较好的给出种植策略,并具有较强的稳定性。

表 2 对比结果分析表

	问题二	问题三	增长率
样本中最大收益值最大值	48786287.63862544	65248153.58262825	0.337428133
样本中最大收益值平均值	41443256.61561046	56284562.23856498	0.337428133
样本中最大收益值标准差	1381606.055269333	18654278.68457913	12.50187965
样本中最大收益值最小值	40102122.969087206	49863578.85694532	0.243414941

由上表可以看出,综合考虑农作物之间的相关性和市场变化的情况下,该模型得到的种植策略能够更灵活地应对不确定性,并在多个方面实现更高的经济效益。相较于问题二,问题三的策略不仅提高了总收益,也优化了成本控制和风险管理,为乡村的农业发展提供了更为科学和全面的解决方案。

八、 模型的评价及推广

8.1 模型的优点

- 1、模型灵活,适应性强:问题一最终的优化模型考虑了市场供需变化和价格波动(如±1%的浮动),能够根据市场情况灵活调整不同作物的种植和销售,模拟了现实中的不确定性。
- 2、模型统一,通用性强:本文构建的非线性规划模型具有较强通用性,针对不同情况适当修改目标函数以及约束条件即可,具有较强推广性;蒙特卡洛方法能够处理最优种植问题中多种不确定性源,且每次运行是相互独立,具有良好通用性。

8.2 模型的不足

- 1、计算成本高,效率低:蒙特卡洛方法需要大量随机样本以获得较为精确的结果,导致计算成本和时间较高收敛速度慢。
- 2、缺乏对农作物特点的深入分析和分类处理:没有以农作物类别为单位建模,以更精准地反映不同类别作物的特性和需求。

8.3 模型的推广

本文中的基于蒙特卡洛方法建立的非线性规划模型具有处理复杂问题和不确定性的强大能力,可在金融投资组合优化、可再生能源规划、供应链管理等领域尽显推广。

九、灵敏度分析

9.1.1. 问题一：微小系数 δ 灵敏度分析

在构建问题一的约束条件时，我们设定了一个关键参数：最小作物面积，具体值为 δX_i ，这一系数因地块类型的差异而有所变化。为了全面评估该参数对模型输出的影响，我们进行灵敏度分析，通过调整每一种地块上的最小作物面积系数，并施加不同的约束条件，系统地观察并记录了面积与利润指标随之变化的趋势。这一分析不仅揭示了最小作物面积设定对农业经济效益的潜在影响，还为后续的优化决策提供了有力支持。

9.2.2. 问题二：风险调整系数 r_i^k 灵敏度分析

针对问题二中的复杂情境，我们引入了预测风险 r_i^k 作为关键变量，并通过结合随机模拟技术来应对每年可能遭遇的极端天气变化。在此框架下，风险调整系数 r_i^k 被设定为不超过 0.3 的上限，以确保模型的合理性与稳健性。为了深入理解 r_i^k 对模型结果的具体作用，我们实施了深入的灵敏度分析。通过模拟华北山区特有的环境风险，并为不同类型的灾害分配相应的风险系数值，我们成功地量化了 r_i^k 的变动如何影响决策的准确性和有效性。

9.3.3. 问题三替代效应修正因子 θ_{jm} 和互补性修正因子 φ_{jm} 灵敏度分析

在问题三的模型构建阶段，我们创新性地引入了替代效应修正因子 θ_{jm} 和互补性修正因子 φ_{jm} ，以精确刻画不同作物之间的相互作用关系。为了验证这两个新引入参数的合理性和有效性，我们同样进行了全面的灵敏度分析。通过不断调整修正因子的取值，我们模拟了多种可能的作物组合与配置情况，并据此评估了这些变化对模型预测结果的改善程度。这一过程不仅增强了模型的灵活性和适应性，也为农业生产实践的优化调整提供了科学依据。

十、参考文献

- [1] 张宁,文慧子.乡村振兴战略下农业可持续发展的问题与对策研究[J].农业经济,2023(11):3-6.
- [2] 刘瑜.农作物种植及病虫害防治技术探究[J].河南农业,2024(16):10-12.
- [3] 郭梁,Andreas Wilkes,于海英,许建初.中国主要农作物产量波动影响因素分析[J].植物分类与资源学报,2013,35(04):513-521.
- [4] 贺大兴.极端气候对中国粮食产量影响的定量分析[J].中国农业资源与区划,2017,38(04):28-34.
- [5] 姜启源.数学模型(第三版)[M]. 高等教育出版社,2003.

附录

附录 1

介绍：支撑材料的文件列表

result:

result1_1.xlsx

result1_2.xlsx

result2.xlsx

问题一:

Problem1_1.py

Problem1_2.py

地块作物属性表.xlsx

种植面积 11.xlsx

种植面积 22.xlsx

作物属性表

问题二:

Forecasted_Data_for_2024-2030.xlsx

problem2.py

result_with_ijk.xlsx

问题三:

问题 3.py

相关性分析.py

附录 2

介绍：问题一第一小问代码（python）

```
import pandas as pd
```

```
import pulp
```

```
# 读取 Excel 文件
```

```
file_path = "地块作物属性表.xlsx"
```

```
df = pd.read_excel(file_path, engine='openpyxl')
```

```
k=range(1,8)
```

```
# 初始化参数字典
```

```
Nij = {}
```

```
Cij = {}
```

```
Xi = {}
```



```

Pj = {}
Yj = {}
# 读取 Excel 数据, 给定存在的组合
for index, row in df.iterrows():
    i = row['i']
    j = row['j']
    Nij[(i, j)] = row['亩产量 (Nij)']
    Cij[(i, j)] = row['种植成本 (Cij)']
    Xi[i] = row['地块面积 (Xi)']
    Pj[j] = row['销售单价均值 (Pj)']
    Yj[j] = row['预期销售量 (Yj)']

# 初始化线性规划问题
model = pulp.LpProblem("Maximize_Profit", pulp.LpMaximize)

# 定义决策变量 Xijk, 总共 54 块地, 41 个农作物, 7 年
X = pulp.LpVariable.dicts("X", ((i, j, k) for i in range(1, 55)
                                for j in range(1, 42)
                                for k in range(1, 8)), lowBound=0, cat='Continuous')

# 定义二进制变量 Zij, 表示某块地是否种植某种作物
Z = pulp.LpVariable.dicts("Z", ((i, j, k) for i in range(1, 55)
                                for j in range(1, 42)
                                for k in range(1, 8)), cat='Binary')

# 为未指定的组合赋默认值 0
for i in range(1, 55):
    for j in range(1, 42):
        if (i, j) not in Nij:
            Nij[(i, j)] = 0
        if (i, j) not in Cij:
            Cij[(i, j)] = 0
        if j not in Pj:
            Pj[j] = 0
        if j not in Yj:
            Yj[j] = 0
    if i not in Xi:
        Xi[i] = 0

```

```

# 添加目标函数
model += pulp.lpSum(X[(i, j, k)] * Pj.get(j, 0) * Nij.get((i, j), 0) - X[(i, j, k)] * Cij.get((i, j), 0)
                    for j in range(1, 42)
                    for i in range(1, 55)
                    for k in range(1, 8))

# 添加约束条件:

# 1. 不能连续种植相同作物, 使用二进制变量 Z 来避免非线性乘积
for k in range(1, 7):
    for i in range(1, 55):
        for j in range(1, 42):
            if j in range(1, 17): # 防止超过 k+1 的范围
                model += Z[(i, j, k)] + Z[(i, j, k+1)] <= 1 # 不允许连续两年种植相同作物 # 不允许
连续两年种植相同作物

# 2. 三年内至少种一次豆类作物 (假设豆类作物的编号为 1,2,3,4,5,17,18,19)
for i in range(1, 55):
    for k in range(1, 6):
        # 防止超过 k+3 的范围
        model += pulp.lpSum([Z[(i, j, k+l)] for l in range(3) for j in (1,2,3,4,5,17,18,19)]) >= 1

# 3. 每块地的总种植面积不能超过总面积
for i in range(1, 55):
    for k in range(1, 8):
        model += pulp.lpSum([X[(i, j, k)] for j in range(1, 42)]) <= Xi[i]

# 4. 种植面积与 Z 变量的关联
# 如果种植了作物 (Z=1), 则面积 X >= 0.5 * 地块总面积, 否则 X = 0

for i in range(1, 55): # 对于所有地块
    for j in range(1, 42): # 对于所有作物
        for k in range(1, 8): # 对于每一年
            # 1. 如果种植了作物 (Z = 1), 则面积 X 应满足  $1/2 * Xi \leq X \leq Xi$ 
            model += X[(i, j, k)] <= Z[(i, j, k)] * Xi[i], f"Max_area_plot_{i}_crop_{j}_year_{k}" # 如
果 Z = 0, X = 0

```



```

        model += X[(i, j, k)] >= Z[(i, j, k)] * 0.5 * Xi[i], f"Min_area_plot_{i}_crop_{j}_year_{k}" #
Z = 1 时,  $X \geq 0.5 * X_i$ 
# 5.添加销售量限制的约束

for j in range(1, 42): # 对于每种作物
    for k in range(1, 8): # 对于每个年份
        model += pulp.lpSum(X[(i, j, k)] * Nij.get((i, j), 0) for i in range(1, 55)) >= Yj.get(j, 0),
f"Sales_constraint_crop_{j}_year_{k}"

#添加转化因素 m
m=4
# 求解模型
model.solve()
# 检查每地的总种植面积是否超过地块面积
for i in range(1, 55):
    for k in range(1, 8):
        total_area_used = sum(X[(i, j, k)].varValue for j in range(1, 42))
        if total_area_used > (Xi[i]):
            print(f"Warning: 地块 {i} 在第 {k} 年的总种植面积 {total_area_used} 超过了总面
积 {Xi[i]}")

# 生成每年 (2024-2030) 种植面积的 Excel 输出
with pd.ExcelWriter('种植面积 11.xlsx', engine='openpyxl') as writer:
    for k in range(1, 8): # k=1 对应 2024 年, 依次到 k=7 对应 2030 年
        # 创建 DataFrame 存储每年各地块和作物的种植面积
        data = {}
        for j in range(1, 42):
            data[f"Crop_{j}"] = [X[(i, j, k)].varValue if X[(i, j, k)].varValue > 0 else 0 for i in range(1,
55)]

        df_year = pd.DataFrame(data, index=[f"Plot_{i}" for i in range(1, 55)])

        # 将每年的 DataFrame 写入不同的 Sheet 中
        df_year.to_excel(writer, sheet_name=f"Year_{2023 + k}")
W = abs(model.objective.value()*m)
print(f"最大收益 W: {W}")

```

附录 3

介绍：问题二代码（python）

```
import numpy as np
import pandas as pd
from scipy.optimize import linprog
import matplotlib.pyplot as plt

# Step 1: 导入数据
data = pd.read_excel('1.xlsx')

# 提取必要的列
i_values = data.iloc[:, 0] # 第 i 列
land_names = data.iloc[:, 1] # 地块名称
land_types = data.iloc[:, 2] # 地块类型
j_values = data.iloc[:, 3] # 第 j 列 (作物编号)
crop_names = data.iloc[:, 4] # 作物名称
seasonal_cycles = data.iloc[:, 5] # 种植季次
land_areas = data.iloc[:, 6] # 地块面积
expected_sales = data.iloc[:, 7] # 预期销售量
sale_price_means = data.iloc[:, 8] # 销售单价均值
yield_per_area = data.iloc[:, 9] # 亩产量
planting_costs = data.iloc[:, 10] # 种植成本

# Step 2: 定义随机变量分布并包括风险调整系数  $r_l^k$ 
def generate_random_variables(j, land_type):
    # 销售量增长率
    if j in [6, 7]: # 小麦和玉米
        sales_growth_rate = np.random.uniform(0.05, 0.1)
    else: # 其他作物
        sales_growth_rate = np.random.beta(1, 0.5) * 0.1 - 0.05

    # 产量增长率
    yield_growth_rate = np.random.beta(1, 0.5) * 0.2 - 0.1

    # 种植成本增长率
    cost_growth_rate = np.random.normal(0.05, 0.01)

    # 销售价格增长率
    if j in range(1, 17): # 蔬菜类作物
```



```

        price_growth_rate = np.random.normal(0.05, 0.01)
    elif j in range(38, 49): # 食用菌价格
        price_growth_rate = np.random.uniform(-0.05, 0.05)
    elif j == 41: # 羊肚菌价格
        price_growth_rate = np.random.normal(0.05, 0.01)
    else:
        price_growth_rate = np.random.normal(0.05, 0.01) # 默认设置

# 风险调整系数  $r_l^k$ 
if land_type == "梯田" or land_type == "水浇地":
    risk_adjustment_factor = np.random.uniform(0, 0.1) # 干旱影响较小
elif land_type == "平坦地":
    risk_adjustment_factor = np.random.uniform(0, 0.3) # 暴雨影响较大
else:
    risk_adjustment_factor = np.random.uniform(0, 0.2) # 其他情况

    return sales_growth_rate, yield_growth_rate, cost_growth_rate, price_growth_rate,
    risk_adjustment_factor

# Step 3: 定义约束函数
def add_constraints(n, areas, yields, sales, costs, prices):
    constraints = []
    bounds = []

    for idx in range(len(i_values)-1):
        constraints.append({
            'type': 'ineq',
            'fun': lambda x, idx=idx: x[idx] * x[idx+1] == 0 # 确保同一作物不能连续两年种植在
            同一地块上
        })

    for idx in range(len(i_values)):
        if j_values[idx] in [1, 2, 3, 4, 5, 17, 18, 19]: # 豆类作物
            constraints.append({
                'type': 'ineq',
                'fun': lambda x, idx=idx: x[idx] + x[idx+1] + x[idx+2] > 0 # 在三年内至少一次种
                植豆类作物
            })

```

```

    })

    for idx in range(len(i_values)):
        constraints.append({
            'type': 'ineq',
            'fun': lambda x, idx=idx: x[idx] * yields[idx] <= sales[idx] # 确保产量不超过预期销售量
        })

    for i in set(i_values):
        constraints.append({
            'type': 'ineq',
            'fun': lambda x, i=i: np.sum([x[idx] for idx in range(len(i_values)) if i_values[idx] == i])
            <= areas[i_values.index(i)]
        })

    # 要么不种植, 要么种植面积至少占该地块面积的一定比例 ( $\delta$ )
    delta = 0.5 # 根据问题中的要求设置比例
    for i in set(i_values):
        constraints.append({
            'type': 'ineq',
            'fun': lambda x, i=i: np.sum([x[idx] for idx in range(len(i_values)) if i_values[idx] ==
            i]) >= delta * areas[i_values.index(i)] or np.sum([x[idx] for idx in range(len(i_values)) if
            i_values[idx] == i]) == 0 # 确保种植面积 >= 该地块面积的一定比例或不种植
        })

    return constraints, bounds

# Step 4: 蒙特卡洛模拟
def monte_carlo_simulation(n_scenarios=1000):
    total_revenue_list = []
    optimal_areas = []

    for _ in range(n_scenarios):
        total_revenue = 0
        current_areas = np.zeros(len(i_values))

```



```

# 随机生成变量
adjusted_sales = np.zeros(len(i_values))
adjusted_yields = np.zeros(len(i_values))
adjusted_costs = np.zeros(len(i_values))
adjusted_prices = np.zeros(len(i_values))
risk_factors = np.zeros(len(i_values))

for idx in range(len(i_values)):
    land_type = land_types[idx] # 根据地块类型生成风险系数
    sales_growth_rate, yield_growth_rate, cost_growth_rate, price_growth_rate,
risk_adjustment_factor = generate_random_variables(j_values[idx], land_type)

    # 对应随机变量的调整值
    adjusted_sales[idx] = expected_sales[idx] * (1 + sales_growth_rate)
    adjusted_yields[idx] = yield_per_area[idx] * (1 + yield_growth_rate)
    adjusted_costs[idx] = planting_costs[idx] * (1 + cost_growth_rate)
    adjusted_prices[idx] = sale_price_means[idx] * (1 + price_growth_rate)
    risk_factors[idx] = risk_adjustment_factor # 风险系数

# 定义目标函数：收益最大化，并考虑风险调整系数
c = -1 * ((adjusted_sales * adjusted_yields * adjusted_prices - adjusted_costs *
adjusted_yields) * (1 - risk_factors))

# 添加约束
constraints, bounds = add_constraints(len(i_values), land_areas, adjusted_yields,
adjusted_sales, adjusted_costs, adjusted_prices)

# 检查是否有解
res = linprog(c, A_ub=None, b_ub=None, bounds=bounds, method='highs')

if res.success: # 确保解是成功的
    optimal_areas.append(res.x)
    total_revenue = -res.fun # 因为目标函数被最小化了，所以需要取负号
    total_revenue_list.append(total_revenue)

return total_revenue_list, optimal_areas

# Step 5: 结果分析并输出稳定的播种面积

```



```

def analyze_results(total_revenue_list, optimal_areas):
    if len(total_revenue_list) == 0: # 如果模拟过程中没有成功的结果，直接返回
        print("没有有效的结果生成，请检查约束条件或随机变量的生成过程。")
        return

    # 统计关键指标
    average_revenue = np.mean(total_revenue_list)
    max_revenue = np.max(total_revenue_list)
    revenue_std = np.std(total_revenue_list)
    best_scenario_idx = np.argmax(total_revenue_list)
    best_areas = optimal_areas[best_scenario_idx]

    # 打印结果
    print(f"平均收益: {average_revenue}")
    print(f"最大收益: {max_revenue}")
    print(f"收益标准差: {revenue_std}")
    print(f"最优情景下的播种面积 (第 k 年, 第 i 地块, 第 j 作物):")
    for idx, area in enumerate(best_areas):
        print(f"第 {i_values[idx]} 地块, 作物 {crop_names[idx]} 播种面积: {area}")

    # 可视化结果
    plt.hist(total_revenue_list, bins=30, edgecolor='k')
    plt.title('收益分布 (Monte Carlo Simulation)')
    plt.xlabel('总收益')
    plt.ylabel('频率')
    plt.show()

# Step 6: 风险评估与敏感性分析
def risk_assessment(total_revenue_list):
    plt.boxplot(total_revenue_list)
    plt.title('收益波动性分析')
    plt.ylabel('总收益')
    plt.show()

# 执行模拟
total_revenue_list, optimal_areas = monte_carlo_simulation(n_scenarios=1000)

# 分析结果

```



```
analyze_results(total_revenue_list, optimal_areas)
```

```
# 风险评估
```

```
    risk_assessment(total_revenue_list)
```

附录 4

介绍：问题三代码（python）

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def read_2023_data(file_path):
```

```
    data = pd.read_excel(file_path, sheet_name='2023') # 读取 2023 年表格
```

```
    return data
```

```
# 模拟参数
```

```
years = list(range(2024, 2031)) # 模拟的年份 (2024-2030)
```

```
simulations = 10000 # 蒙特卡洛模拟次数
```

```
# 读取 2023 年数据
```

```
file_path = '1.xlsx'
```

```
data_2023 = read_2023_data(file_path)
```

```
# 根据读取的数据初始化农作物种类
```

```
crops = data_2023['作物'].values.tolist() # 从 2023 数据中提取作物种类
```

```
planting_area_2023 = data_2023['种植面积'].values.tolist() # 提取 2023 种植面积数据
```

```
yield_2023 = data_2023['亩产量'].values.tolist() # 提取 2023 亩产量数据
```

```
cost_2023 = data_2023['种植成本'].values.tolist() # 提取 2023 种植成本数据
```

```
price_2023 = data_2023['销售价格'].values.tolist() # 提取 2023 销售价格数据
```

```
# 初始化参数
```

```
theta_jm = 0.9 # 替代效应系数, 可根据实际问题调整
```

```
phi_jm = 0.1 # 互补修正因子
```

```
best_Xijk = None # 初始化最佳种植方案
```

```
max_profit = -np.inf # 初始化最大收益
```



```

# 模拟随机变量的分布
crop_sales_growth_rate = [np.random.uniform(0.05, 0.10) if j in [6, 7] else (0.10 *
np.random.binomial(1, 0.5) - 0.05) for j in range(len(crops))] # 销售增长率
crop_yield_growth_rate = [0.20 * np.random.binomial(1, 0.5) - 0.10 for _ in range(len(crops))] #
亩产量增长率
crop_cost_growth_rate = [np.random.normal(0.05, 0.01) for _ in range(len(crops))] # 种植成本
增长率
price_growth_rate = [np.random.normal(0.05, 0.01) if j <= 16 else np.random.uniform(-0.05, -
0.01) if j >= 38 and j <= 49 else np.random.normal(-0.05, 0.01) if j == 41 else 0 for j in
range(len(crops))] # 销售价格增长率

# 天气事件及其影响
extreme_events = ['干旱', '沙尘暴', '暴雪', '暴雨']
event_probabilities = [0.25] * 4 # 各种极端天气事件的概率相等
no_event_prob = 0.5 # 没有极端天气事件的概率
risk_adjustment_max = 0.3 # 风险调整系数最大值

# 模拟函数
def simulate_weather_risk():
    if np.random.rand() < no_event_prob:
        return 0 # 无风险调整
    event = np.random.choice(extreme_events, p=event_probabilities)
    return np.random.uniform(0, risk_adjustment_max) # 返回随机风险调整系数

def monte_carlo_simulation():
    global best_Xijk, max_profit

    for sim in range(simulations):
        total_profit = 0
        Xijk = np.zeros((len(crops), len(planting_area_2023), len(years))) # 初始化种植方案

        for k, year in enumerate(years):
            risk_factor = simulate_weather_risk() # 每一年模拟极端天气和风险调整系数

            for i, crop in enumerate(crops):
                sales_growth = crop_sales_growth_rate[i]
                yield_growth = crop_yield_growth_rate[i] - risk_factor

```



```

cost_growth = crop_cost_growth_rate[i]
price_growth = price_growth_rate[i]

# 使用 2023 年的初始数据进行模拟
planting_area = planting_area_2023[i]
yield_per_acre = yield_2023[i] * (1 + yield_growth)
cost_per_acre = cost_2023[i] * (1 + cost_growth)
price_per_unit = price_2023[i] * (1 + price_growth)

# 计算每年农作物的收益，加入替代效应和互补性修正
revenue = planting_area * yield_per_acre * price_per_unit * theta_jm
cost = planting_area * cost_per_acre * phi_jm
adjusted_profit = (revenue - cost) * (1 - risk_factor)

total_profit += adjusted_profit

# 为此年此作物保存最佳种植面积
Xijk[i, i, k] = planting_area

# 记录最大收益方案
if total_profit > max_profit:
    max_profit = total_profit
    best_Xijk = Xijk

return max_profit, best_Xijk

# 运行蒙特卡洛模拟
max_profit, best_Xijk = monte_carlo_simulation()

# 输出最佳种植方案
print(f"最大收益: {max_profit:.2f}")
print(f"最佳种植方案 (Xijk): {best_Xijk}")

# 绘制模拟结果
fig, ax = plt.subplots(figsize=(10, 6))
for i, crop in enumerate(crops):
    ax.plot(years, best_Xijk[i, i, :], label=f'{crop} 种植面积')
ax.set_title("最佳种植方案每年的种植面积")

```

```
ax.set_xlabel("年份")  
ax.set_ylabel("种植面积")  
ax.legend()  
plt.show()
```